```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.metrics import roc_auc_score

# 1. Данные
df = pd.read_csv("https://api.vitaldb.net/cases")
labs = pd.read_csv("https://api.vitaldb.net/labs")
df["opdur"] = (df["opend"] - df["opstart"]) / 60

# 2. AKI по KDIGO
cr = labs[labs["name"] == "cr"]
day = 24 * 3600
p7 = cr[(cr["dt"] > 0) & (cr["dt"] <= 7 * day)]
p48 = cr[(cr["dt"] > 0) & (cr["dt"] <= 2 * day)]
mx7 = p7.groupby("caseid")["result"].max()
mx48 = p48.groupby("caseid")["result"].max()

c = df.set_index("caseid")
c["max7"] = mx7
c["max48"] = mx48
c["post"] = c["max7"].notna()
r7 = c["max7"] >= 1.5 * c["preop_cr"]
r48 = (c["max48"] - c["preop_cr"]) >= 0.3
c["aki"] = (r7 | r48).astype(int)

# 3. Отбор (схема для статьи)
kw = "nephr|kidney|ureter|transplant|donor|cystectomy"
need = ["preop_dm", "preop_hb", "preop_alb",
        "preop_na", "asa", "emop", "preop_cr"]
st = [("все операции", len(c))]
c = c[c["age"] >= 18]
st.append(("взрослые", len(c)))
c = c[c["opdur"] >= 60]
st.append(("операция >= 60 мин", len(c)))
c = c.reset_index().drop_duplicates("subjectid")
c = c.set_index("caseid")
st.append(("1 операция на пациента", len(c)))
bad = c["opname"].str.contains(kw, case=False, na=False)
c = c[~bad]
st.append(("без почек и трансплантаций", len(c)))
c = c.dropna(subset=need)
st.append(("полные данные", len(c)))
```

```
fe = c["sex"] == "F"
k = np.where(fe, 0.7, 0.9)
a = np.where(fe, -0.241, -0.302)
r = c["preop_cr"] / k
g = 142 * np.minimum(r, 1) ** a
g = g * np.maximum(r, 1) ** -1.2
g = g * 0.9938 ** c["age"]
c["egfr"] = g * np.where(fe, 1.012, 1)
c = c[c["egfr"] >= 15]
st.append(("СКФ >= 15 (анализ чувствит.)", len(c)))
sens = c.copy()
c = c[c["post"]]
st.append(("есть креатинин после (основной)", len(c)))
for nm, n in st:
    print(f"{nm:34s} {n}")


# 4. Модели
def models(c):
    f = (c["sex"] == "F").astype(int)
    m = 1 - f
    age = c["age"]
    eg = c["egfr"]
    dur = c["opdur"] / 60
    dm = c["preop_dm"]
    em = c["emop"]
    asa = c["asa"]
    hb = c["preop_hb"]
    an = ((f == 1) & (hb < 12)) | ((f == 0) & (hb < 13))
    an = an.astype(int)
    alb = (c["preop_alb"] < 3.5).astype(int)
    na = (c["preop_na"] < 135).astype(int)
    a1 = ((age >= 40) & (age < 60)).astype(int)
    a2 = ((age >= 60) & (age < 80)).astype(int)
    a3 = (age >= 80).astype(int)
    e0 = (eg >= 60).astype(int)
    e1 = ((eg >= 45) & (eg < 60)).astype(int)
    e2 = ((eg >= 30) & (eg < 45)).astype(int)
    e3 = ((eg >= 15) & (eg < 30)).astype(int)
    op = (c["approach"] == "Open").astype(int)
    s1 = (asa == 1).astype(int)
    s2 = (asa == 2).astype(int)
    s3 = (asa == 3).astype(int)
    s4 = (asa >= 4).astype(int)
    X = {}
```

```
    X["Park"] = (-5.9651 + 0.522*a1 + 0.852*a2
        + 1.203*a3 + 0.705*m + 0.347*dm
        + 0.459*dur + 0.678*em + 0.690*e1
        + 1.345*e2 + 2.012*e3 + 0.319*an
        + 0.705*alb + 0.298*na)
    X["Nishimoto"] = (-3.853581 + 0.272*a1
        + 0.178*a2 + 0.064*a3 + 0.341*m
        + 0.238*dm + 0.032*dur + 0.012*em
        + 0.446*e1 + 1.143*e2 + 1.267*e3
        + 0.399*an + 0.705*alb + 0.071*na)
    X["Nepogodiev"] = (-1.102349 + 0.017026*age
        + 0.280635*m - 0.052892*eg
        + 0.000295*eg**2 + 0.274972*op
        + 0.231126*s2 + 0.371917*s3
        + 1.034723*s4)
    X["Bell"] = (-2.385 + 0.022*age - 0.708*f
        + 0.427*dm - 1.417*e0 - 1.108*e1
        - 0.676*e2 - 1.037*s1 - 0.141*s2
        + 0.014*s3)
    return X


# 5. Точность, интервалы, калибровка
def report(c, title):
    X = models(c)
    y = c["aki"].astype(int).values
    print()
    print(title, "| n =", len(y), "| AKI =", y.sum(),
          f"({y.mean():.1%})")
    rng = np.random.default_rng(42)
    B = sm.families.Binomial()
    for n in X:
        lp = X[n].values
        p = 1 / (1 + np.exp(-lp))
        auc = roc_auc_score(y, p)
        bs = []
        for i in range(1000):
            ix = rng.integers(0, len(y), len(y))
            if 0 < y[ix].sum() < len(ix):
                bs.append(roc_auc_score(y[ix], p[ix]))
        lo, hi = np.percentile(bs, [2.5, 97.5])
        one = np.ones(len(y))
        m0 = sm.GLM(y, one, family=B, offset=lp).fit()
        m1 = sm.GLM(y, sm.add_constant(lp), family=B).fit()
        print(f"{n:11s} AUROC {auc:.3f} ({lo:.3f}-{hi:.3f})"
              f"  прогноз {p.mean():.1%}"
```

```
            f"  O/E {y.mean() / p.mean():.2f}"
            f"  CITL {m0.params[0]:.2f}"
            f"  slope {m1.params[1]:.2f}")
    return X


X = report(c, "ОСНОВНОЙ АНАЛИЗ")
Xs = report(sens, "АНАЛИЗ ЧУВСТВИТЕЛЬНОСТИ")
```